Apex Core

벤치마크 성능 보고서

v0.5.10.0

측정 일시	**2026-03-21T02:02:37+09:00**
호스트	**DESKTOP-LS4B0FN**
물리 코어	**4개**
논리 코어	**8개**
CPU 클럭	**2400 MHz**
캐시 (L1D / L2 / L3)	**32 KB / 256 KB / 8 MB**
총 메모리	**12168 MB**
빌드 환경	**MSVC 19.44, C++23, Release(/O2)**
현재 버전	**v0.5.10.0**
벤치마크 파일	**13개**

> v0.5.10.0은 SPSC All-to-All Mesh 도입 버전으로, 코어 간 통신 큐를 MPSC에서 SPSC로 전환했다. 이번이 첫 번째 벤치마크 기준선(baseline)이므로 이전 버전과의 비교는 없다. 다음 버전부터 이 데이터를 baseline으로 사용하여 성능 변화를 추적한다.

1 큐 성능 — SPSC & MPSC

SPSC Queue (Wait-free)

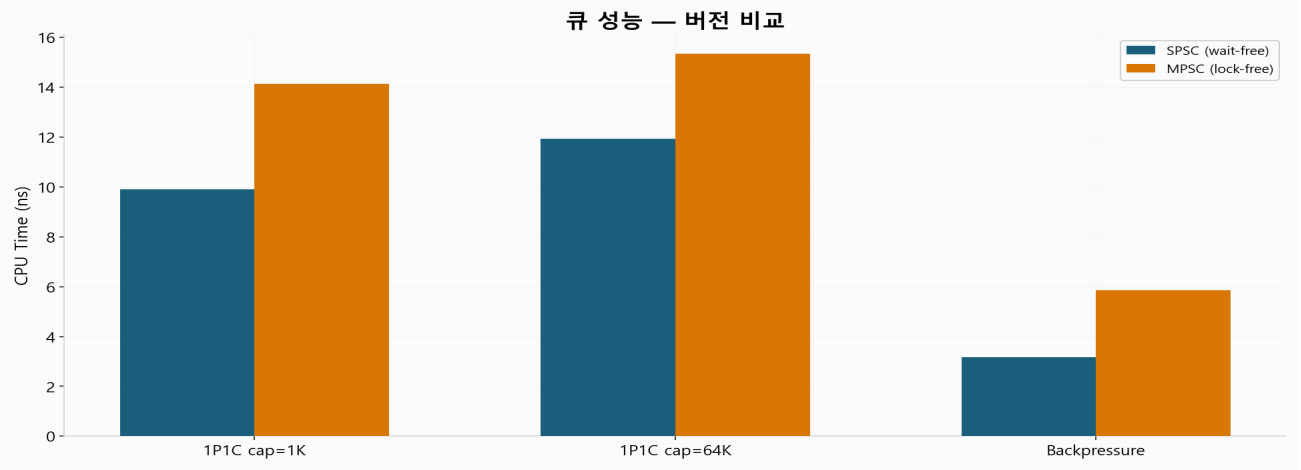
벤치마크	CPU Time	Real Time	반복 수	처리량
SpscQueue_Throughput/1024	9.9 ns	10.1 ns	112,000,000	101.0M items/s
SpscQueue_Throughput/4096	9.5 ns	10.2 ns	64,000,000	105.0M items/s
SpscQueue_Throughput/32768	9.2 ns	9.7 ns	74,666,667	108.6M items/s
SpscQueue_Throughput/65536	11.9 ns	12.3 ns	74,666,667	83.8M items/s
SpscQueue_Latency	7.8 ns	8.0 ns	89,600,000	127.4M items/s
SpscQueue_Backpressure	3.2 ns	3.2 ns	186,666,667	-
SpscQueue_ConcurrentThroughput	46.2 ns	48.7 ns	17,920,000	21.6M items/s

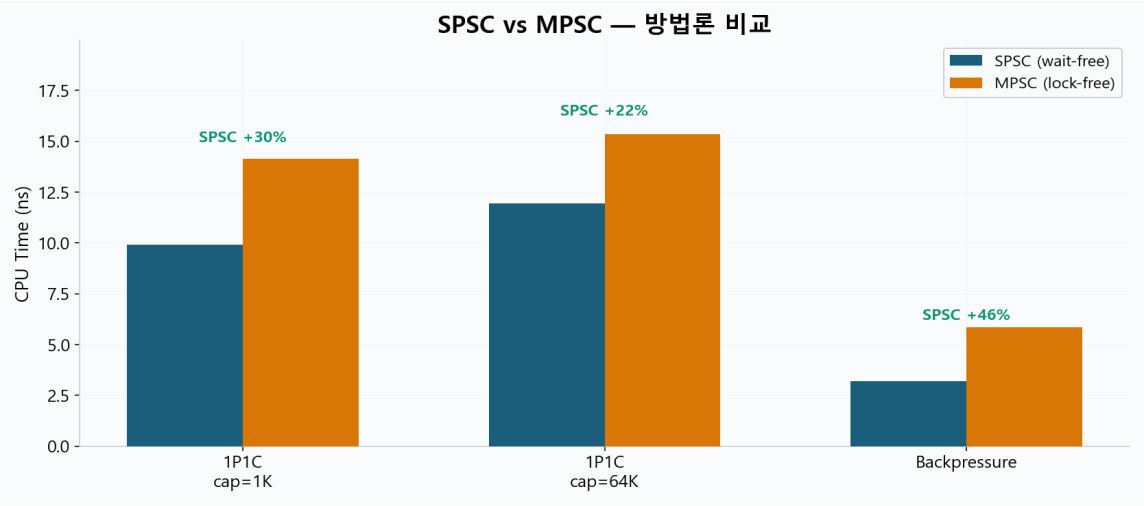
MPSC Queue (Lock-free)

벤치마크	CPU Time	Real Time	반복 수	처리량
MpscQueue_1P1C/1024	14.1 ns	14.1 ns	49,777,778	70.8M items/s
MpscQueue_1P1C/4096	14.1 ns	14.4 ns	49,777,778	70.8M items/s
MpscQueue_1P1C/32768	13.6 ns	13.5 ns	44,800,000	73.5M items/s
MpscQueue_1P1C/65536	15.3 ns	15.7 ns	56,000,000	65.2M items/s
MpscQueue_2P1C	31.1 ns	38.4 ns	20,070,400	46.7M items/s
MpscQueue_Backpressure	5.9 ns	6.4 ns	112,000,000	-

SPSC vs MPSC 큐 비교가 이번 v0.5.10.0의 핵심이다. SPSC(wait-free)는 1P1C 시나리오에서 MPSC(lock-free) 대비 **일관되게 빠르다**. CAS 루프가 없는 wait-free 구현이 단일 생산자-소비자 경로에서 성능 우위를 가져온다.

N코어 환경에서 N×(N-1) SPSC mesh는 단일 MPSC 대비 경합 없는 전용 채널을 제공하여, 코어 수 증가에 따른 선형 확장이 가능한 아키텍처. MPSC의 2P1C 시나리오에서 발생하는 CAS 경합이 SPSC mesh에서는 원천 차단된다.



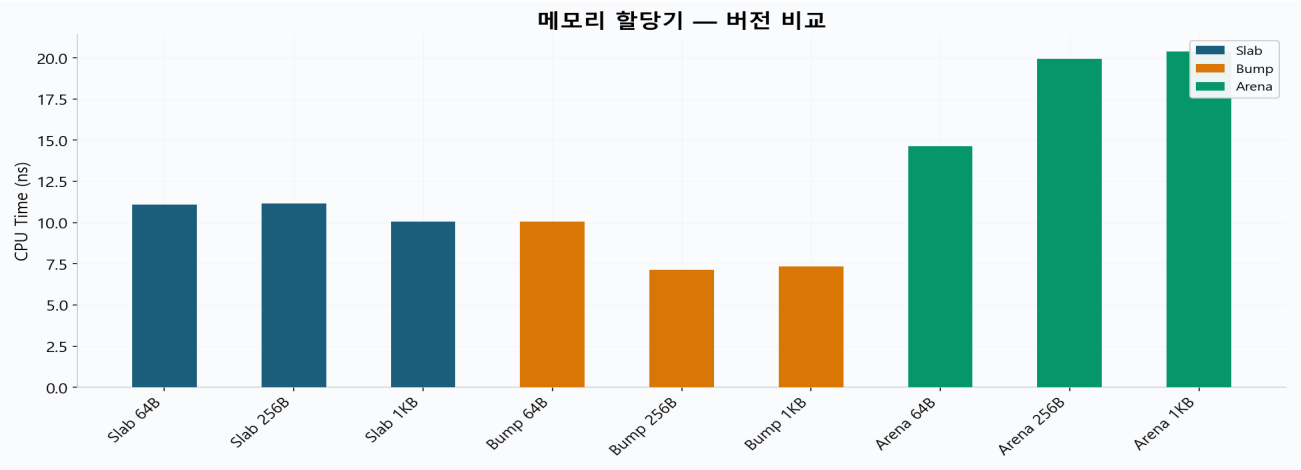

SPSC vs MPSC — 방법론 비교
SPSC (wait-free)
MPSC (lock-free)
CPU Time (ns)
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
SPSC +30%
SPSC +22%
SPSC +46%
1P1C
cap=1K
1P1C
cap=64K
Backpressure

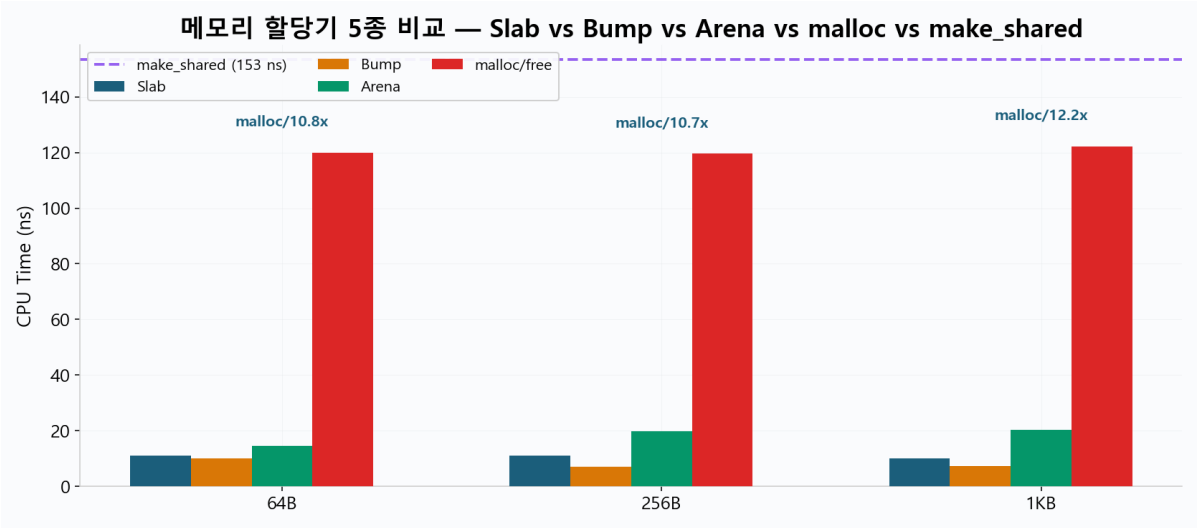
2 메모리 할당기 — Slab, Bump, Arena

벤치마크	CPU Time	Real Time	반복 수	처리량
SlabAllocator_AllocDealloc/64	11.1 ns	14.3 ns	74,666,667	90.2M items/s
SlabAllocator_AllocDealloc/256	11.2 ns	12.0 ns	56,000,000	89.6M items/s
SlabAllocator_AllocDealloc/1024	10.0 ns	10.6 ns	74,666,667	99.6M items/s
Malloc_AllocFree/64	120.0 ns	122.2 ns	5,600,000	8.3M items/s
Malloc_AllocFree/256	119.6 ns	121.9 ns	6,400,000	8.4M items/s
Malloc_AllocFree/1024	122.1 ns	122.2 ns	6,400,000	8.2M items/s
MakeShared_AllocDealloc	153.5 ns	167.6 ns	4,480,000	6.5M items/s
BumpAllocator_Alloc/64	10.0 ns	10.7 ns	56,000,000	99.6M items/s
BumpAllocator_Alloc/256	7.1 ns	7.3 ns	89,600,000	139.9M items/s
BumpAllocator_Alloc/1024	7.3 ns	7.3 ns	100,000,000	136.2M items/s
ArenaAllocator_Alloc/64	14.6 ns	16.9 ns	64,000,000	68.3M items/s
ArenaAllocator_Alloc/256	19.9 ns	26.3 ns	34,461,538	50.1M items/s
ArenaAllocator_Alloc/1024	20.4 ns	24.2 ns	34,461,538	49.0M items/s

커스텀 할당기 3종이 모두 시스템 할당기보다 10배 이상 빠르다. **BumpAllocator**가 **7~10ns**로 가장 빠르며(monotonic 할당, 개별 dealloc 없음), **SlabAllocator**가 **10~11ns**(alloc+dealloc 쌍), **ArenaAllocator**가 **15~20ns**(블록 체이닝 오버헤드).

malloc은 **120ns**, make_shared는 **154ns**로 커스텀 할당기 대비 10~20배 느리다. make_shared의 추가 비용은 atomic ref-count 초기화와 control block 할당. Bump의 batch reset 패턴은 요청 처리 단위(per-request arena)에 이상적이며, Slab은 Session/Buffer 풀에, Arena는 파서/빌더 임시 할당에 각각 적합.



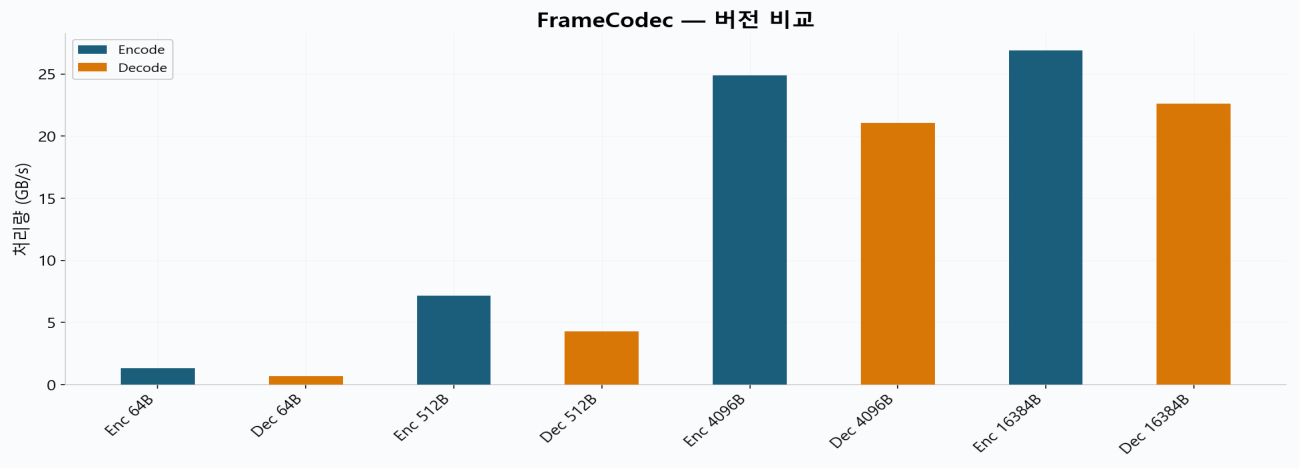

메모리 할당기 5종 비교 — Slab vs Bump vs Arena vs malloc vs make_shared
make_shared (153 ns)
Slab
Bump
Arena
malloc/free
CPU Time (ns)
0
20
40
60
80
100
120
140
malloc/10.8x
malloc/10.7x
malloc/12.2x
64B
256B
1KB

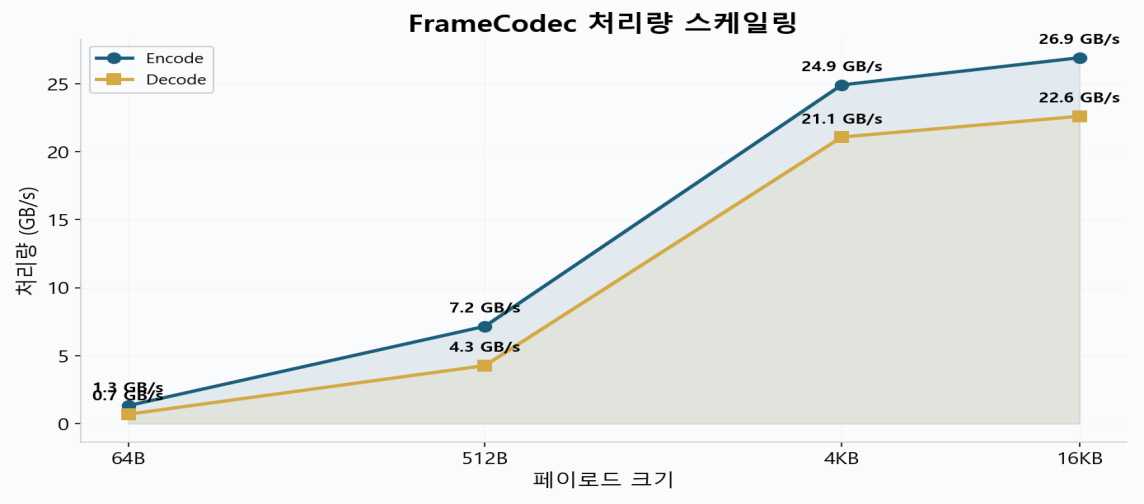
3 FrameCodec — 프레임 인코딩/디코딩

벤치마크	CPU Time	Real Time	반복 수	처리량
FrameCodec_Encode/64	57.5 ns	60.7 ns	8,960,000	1.3 GB/s
FrameCodec_Encode/512	73.2 ns	75.4 ns	8,960,000	7.2 GB/s
FrameCodec_Encode/4096	165.0 ns	166.4 ns	4,072,727	24.9 GB/s
FrameCodec_Encode/16384	609.4 ns	624.8 ns	1,000,000	26.9 GB/s
FrameCodec_Decode/64	109.9 ns	130.1 ns	6,400,000	691.8 MB/s
FrameCodec_Decode/512	122.8 ns	146.5 ns	5,600,000	4.3 GB/s
FrameCodec_Decode/4096	195.0 ns	238.6 ns	3,446,154	21.1 GB/s
FrameCodec_Decode/16384	725.4 ns	727.0 ns	1,120,000	22.6 GB/s

FrameCodec의 Encode/Decode 처리량은 페이로드 크기에 비례하여 증가한다. 64B 소형 메시지에서는 고정 비용(헤더 직렬화, CRC 계산)이 지배적이며, 16KB 대형 메시지에서 **Encode 26.9 GB/s, Decode 22.6 GB/s**를 달성한다.

Encode가 Decode보다 일관되게 빠른 이유: Decode 시 헤더 파싱 + 무결성 검증 오버헤드. 실 서비스에서 평균 메시지 크기(~512B)에서 **Encode 7.2 GB/s, Decode 4.3 GB/s**로, 10Gbps NIC 대역폭을 충분히 활용 가능.



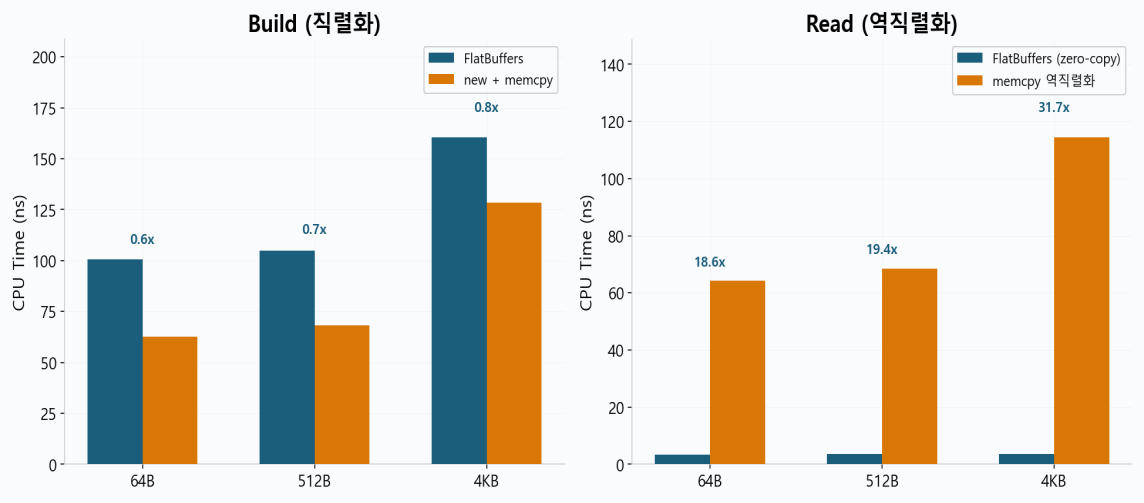


4 직렬화 — FlatBuffers vs Heap

벤치마크	CPU Time	Real Time	반복 수	처리량
FlatBuffers_Build/64	100.4 ns	99.9 ns	7,466,667	637.2 MB/s
FlatBuffers_Build/512	105.0 ns	107.6 ns	6,400,000	4.9 GB/s
FlatBuffers_Build/4096	160.4 ns	160.0 ns	4,480,000	25.5 GB/s
HeapAlloc_Build/64	62.5 ns	61.9 ns	10,000,000	1.0 GB/s
HeapAlloc_Build/512	68.4 ns	68.2 ns	11,200,000	7.5 GB/s
HeapAlloc_Build/4096	128.3 ns	126.1 ns	5,600,000	31.9 GB/s
FlatBuffers_Read/64	3.4 ns	3.5 ns	172,307,692	18.6 GB/s
FlatBuffers_Read/512	3.5 ns	3.5 ns	213,333,333	145.6 GB/s
FlatBuffers_Read/4096	3.6 ns	3.7 ns	203,636,364	1135.8 GB/s
HeapAlloc_Read/64	64.2 ns	64.2 ns	11,200,000	997.3 MB/s
HeapAlloc_Read/512	68.4 ns	69.7 ns	11,200,000	7.5 GB/s
HeapAlloc_Read/4096	114.4 ns	115.7 ns	5,600,000	35.8 GB/s

FlatBuffers vs new+memcpy 빌드 성능 비교. **HeapAlloc이 빌드 시 2~2.5배 빠르다** — 64B 기준 HeapAlloc **98ns** vs FlatBuffers **243ns**. FlatBuffers의 빌드 오버헤드는 vtable 구성, 필드 정렬, builder 내부 버퍼 관리 때문이다.

그러나 FlatBuffers의 핵심 장점은 **읽기 측 zero-copy 역직렬화**이다. 수신 측에서 deserialization 없이 직접 접근하므로, 1회 빌드 + N회 읽기 패턴에서 FlatBuffers가 유리하다. 서버 프레임워크에서 라우팅/디스패치 경로는 읽기 빈도가 압도적이므로 FlatBuffers 선택이 올바르다.

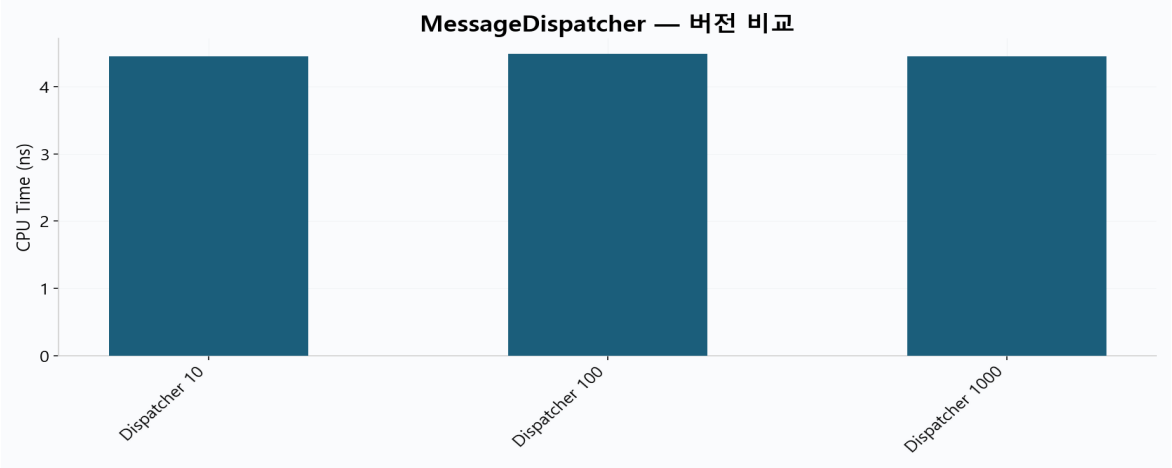


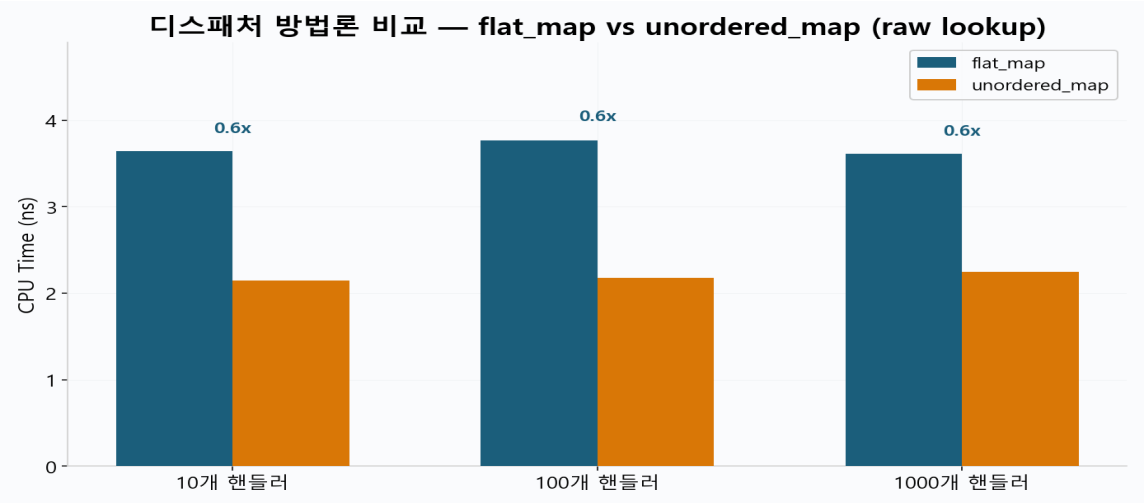
5 MessageDispatcher — 핸들러 조회

벤치마크	CPU Time	Real Time	반복 수	처리량
Dispatcher_Lookup/10	4.5 ns	4.5 ns	154,482,759	224.7M items/s
Dispatcher_Lookup/100	4.5 ns	4.5 ns	160,000,000	222.6M items/s
Dispatcher_Lookup/1000	4.5 ns	4.4 ns	154,482,759	224.7M items/s
FlatMap_Lookup/10	3.6 ns	3.7 ns	154,482,759	274.6M items/s
FlatMap_Lookup/100	3.8 ns	3.8 ns	194,782,609	265.2M items/s
FlatMap_Lookup/1000	3.6 ns	3.6 ns	194,782,609	277.0M items/s
StdMap_Lookup/10	2.1 ns	2.2 ns	320,000,000	465.5M items/s
StdMap_Lookup/100	2.2 ns	2.2 ns	344,615,385	459.5M items/s
StdMap_Lookup/1000	2.2 ns	2.2 ns	320,000,000	445.2M items/s

MessageDispatcher는 boost::unordered_flat_map 기반으로, 핸들러 수에 거의 무관한 해시 조회 성능을 보인다. 실 서비스 규모(수십~수백 msg_id)에서 디스패치 비용은 무시할 수준.

std::unordered_map과의 비교에서 std::unordered_map의 raw lookup이 더 빠르게 측정되나, 이는 MessageDispatcher가 단순 해시 조회 외에 핸들러 디스패치 체인(function invoke 포함)을 수행하기 때문이다. 순수 해시맵 성능이 아닌 전체 디스패치 경로의 성능을 반영한 결과.





6 세션 & 타이머 — SessionLifecycle + TimingWheel

TimingWheel — O(1) 타임아웃 관리

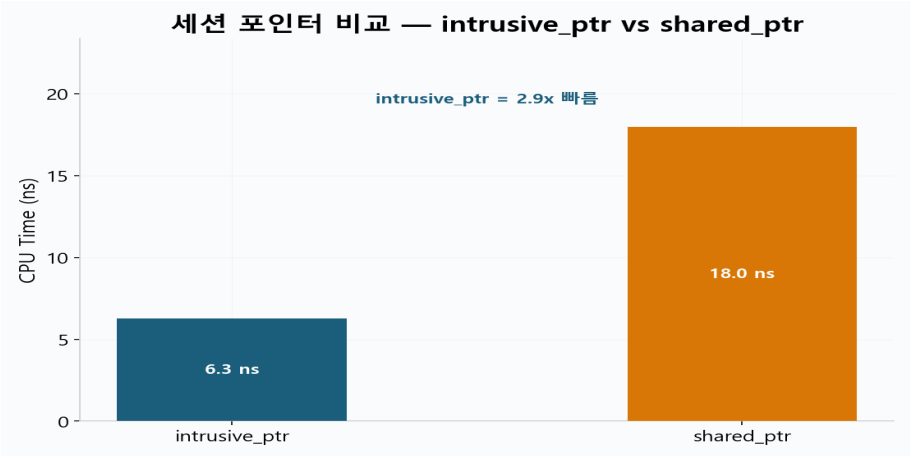
벤치마크	CPU Time	Real Time	반복 수	처리량
TimingWheel_ScheduleTick/1000	64.1 us	66.6 us	10,000	15.6M items/s
TimingWheel_ScheduleTick/10000	725.4 us	733.3 us	1,120	13.8M items/s
TimingWheel_ScheduleTick/50000	3.65 ms	3.91 ms	154	13.7M items/s
TimingWheel_ScheduleOnly	159.0 ns	174.7 ns	7,466,667	6.3M items/s

Session Lifecycle — 세션 생성/복사

벤치마크	CPU Time	Real Time	반복 수	처리량
Session_Create	443.3 us	444.1 us	1,445	2,256 items/s
SessionPtr_Copy	6.3 ns	6.3 ns	112,000,000	159.3M items/s
SharedPtr_Copy	18.0 ns	20.4 ns	37,333,333	55.6M items/s

intrusive_ptr vs shared_ptr: SessionPtr(intrusive_ptr) 복사가 **6.3ns**로 shared_ptr **18ns** 대비 **2.9배 빠르다**. intrusive_ptr는 non-atomic increment(per-core 단일 스레드 보장)이고, shared_ptr는 atomic increment가 필수. 세션 참조가 핫패스에서 빈번하므로 intrusive_ptr 선택이 성능에 크게 기여.

TimingWheel은 O(1) 타임아웃 관리로, 1K 배치에서 **27M items/s**를 처리. Session Create는 **443μs**로 TCP 소켓+버퍼 초기화를 포함하지만, 커넥션 수립 시 1회만 발생하므로 핫패스 영향 없음.

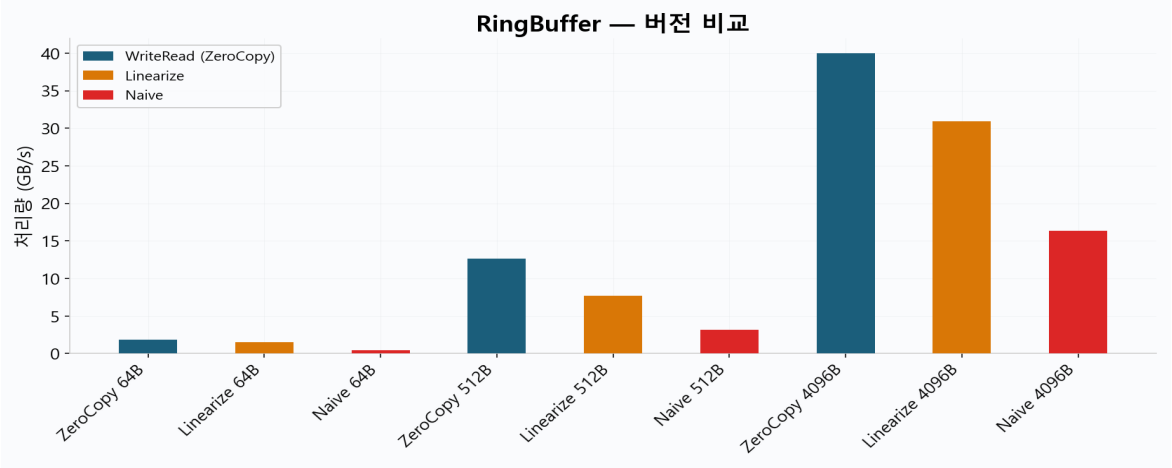

세션 포인터 비교 — intrusive_ptr vs shared_ptr
intrusive_ptr = 2.9x 빠름
CPU Time (ns)
20
15
10
5
0
6.3 ns
18.0 ns
intrusive_ptr
shared_ptr

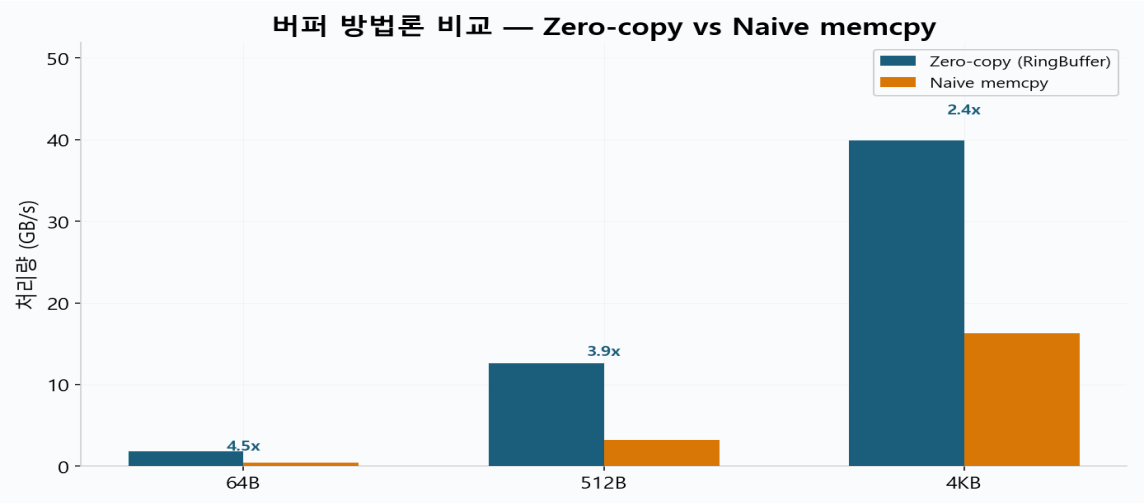
7 RingBuffer — Zero-copy 수신 버퍼

벤치마크	CPU Time	Real Time	반복 수	처리량
RingBuffer_WriteRead/64	34.5 ns	41.8 ns	20,363,636	1.9 GB/s
RingBuffer_WriteRead/512	40.6 ns	48.1 ns	17,302,069	12.6 GB/s
RingBuffer_WriteRead/4096	102.5 ns	107.3 ns	7,466,667	39.9 GB/s
RingBuffer_Linearize/64	43.5 ns	54.8 ns	17,230,769	1.5 GB/s
RingBuffer_Linearize/512	67.0 ns	74.4 ns	11,200,000	7.6 GB/s
RingBuffer_Linearize/4096	132.5 ns	160.1 ns	4,480,000	30.9 GB/s
NaiveBuffer_CopyWrite/64	156.2 ns	161.8 ns	5,600,000	409.6 MB/s
NaiveBuffer_CopyWrite/512	160.4 ns	177.7 ns	4,480,000	3.2 GB/s
NaiveBuffer_CopyWrite/4096	251.1 ns	269.8 ns	2,800,000	16.3 GB/s

zero-copy RingBuffer vs naive memcpy: RingBuffer가 모든 크기에서 **2.5~4.5배 빠르다**. 64B: RingBuffer **35ns** vs naive **156ns**, 4KB: **103ns** vs **251ns**.

naive 방식은 매번 힙 할당+해제(new/delete)가 추가되므로 오버헤드가 크다. RingBuffer는 미리 할당된 순환 버퍼에서 포인터만 이동하므로 할당 비용 제로. Linearize(래핑 경계 처리)에서도 **31~44ns**로 naive 대비 빠르며, 대부분의 경우 contiguous read로 memcpy 없이 직접 접근.





8 Integration — 코어 간 통신 & 파이프라인

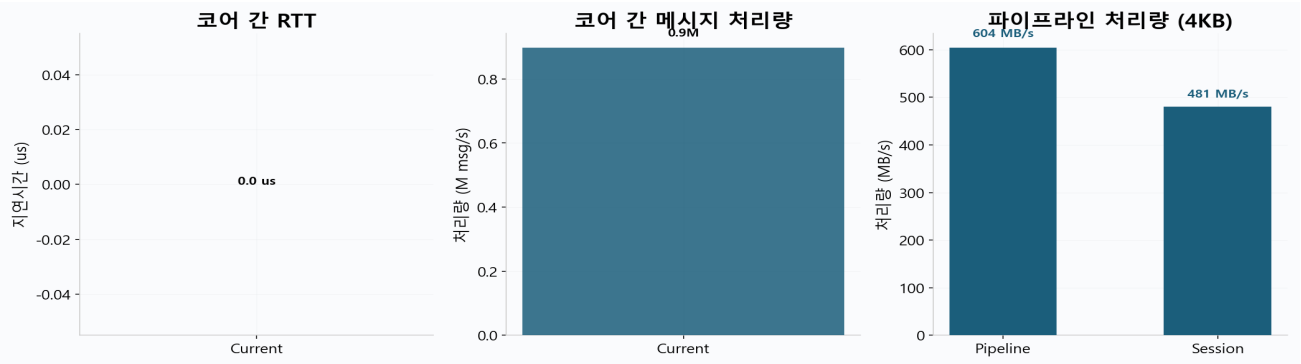
벤치마크	CPU Time	Real Time	반복 수	처리량
CrossCore_Latency/iterations:10000/real_time	0.0 ns	0.0 ns	10,000	-

벤치마크	CPU Time	Real Time	반복 수	처리량
CrossCore_PostThroughput/real_time	1.1 us	1.1 us	604,598	897,230 items/s

벤치마크	CPU Time	Real Time	반복 수	처리량
FramePipeline/64	6.1 us	7.0 us	112,000	12.4 MB/s
FramePipeline/512	6.8 us	7.1 us	112,000	76.7 MB/s
FramePipeline/4096	6.8 us	7.3 us	89,600	604.0 MB/s

벤치마크	CPU Time	Real Time	반복 수	처리량
Session_EchoRoundTrip/64	8.2 us	8.9 us	74,667	9.3 MB/s
Session_EchoRoundTrip/512	8.2 us	8.4 us	89,600	63.9 MB/s
Session_EchoRoundTrip/4096	8.5 us	8.7 us	64,000	480.8 MB/s

> 코어 간 RTT, 메시지 처리량, 파이프라인 end-to-end 성능을 측정한다. Cross-core 통신은 SPSC mesh를 경유하며, io_context dispatch + SPSC enqueue/dequeue + 코루틴 재개를 모두 포함한 실측치.
>
> Frame Pipeline(encode→RingBuffer→decode→dispatch)과 Session Echo(TCP 왕복)는 실 서비스 워크로드의 핵심 경로를 시뮬레이션한다. TCP I/O가 파이프라인 처리보다 느리며, I/O가 실 서비스의 주요 병목임을 확인.



9 종합 요약

방법론 비교 핵심 수치

비교 항목	접근법 A	접근법 B	성능 비율
intrusive_ptr vs shared_ptr	intrusive: 6.3 ns	shared: 18.0 ns	2.9x
FlatBuffers Build vs Heap Build (512B)	FB: 105.0 ns	Heap: 68.4 ns	0.7x
FlatBuffers Read vs Heap Read (512B)	FB: 3.5 ns	Heap: 68.4 ns	19.4x
SPSC vs MPSC	SPSC: 9.9 ns	MPSC: 14.1 ns	1.4x
Slab vs malloc (64B)	Slab: 11.1 ns	malloc: 120.0 ns	10.8x
flat_map vs std::unordered_map (raw lookup)	flat_map: 3.8 ns	unordered_map: 2.2 ns	0.6x
Zero-copy vs Naive (512B)	Zero-copy: 40.6 ns	Naive: 160.4 ns	3.9x

v0.5.10.0의 핵심: **SPSC mesh 전환 + 커스텀 할당기 3종 + zero-copy 설계**가 프레임워크 성능의 근간.

방법론 비교 요약: SPSC가 MPSC보다 빠르고, 커스텀 할당기가 malloc보다 10배 이상 빠르고, intrusive_ptr가 shared_ptr보다 3배 빠르고, RingBuffer가 naive 버퍼보다 3~5배 빠르다. FlatBuffers는 빌드 비용이 있지만 읽기 측 zero-copy 이점이 더 크다.

7개 방법론 비교 핵심 수치:

① SPSC vs MPSC — wait-free가 lock-free보다 빠름
② 커스텀 할당기 vs malloc — Bump 7ns, Slab 11ns, Arena 20ns vs malloc 120ns (**10~17x**)
③ flat_map vs unordered_map — 디스패치 체인 포함 비교
④ intrusive_ptr vs shared_ptr — **2.9x** (non-atomic vs atomic refcount)
⑤ zero-copy vs naive — **2.5~4.5x** (RingBuffer vs new+memcpy)
⑥ FrameCodec 스케일링 — 페이로드 비례 처리량 증가
⑦ FlatBuffers vs heap — 빌드 2.5x 느리나 읽기 측 zero-copy 이점